```
    <% else %>
      <%= seat.number %>
    <% end %>
  </span>
</td>
```

The seat file is worth a little unpacking. We've got a span with the same CSS to draw the border plus a couple of special data attributes, two of which are there to manage the Cypress tests, and the other of which, we'll see, is used by the ActionCable handler. Inside the span, if the seat is available to be purchased, we use the Rails button_to helper to make it a button that submits a form. We use the existing status of the seat to determine whether the form submit is a PUT for a purchase or a DELETE for an unpurchase.

To get the data set up properly, the controller doesn't change, but I did add some methods to the Concert class so that it would create the new Row models:

appendix_hotwire/01/app/models/concert.rb

```
def rows(tickets_to_buy_count: 1)
  tickets.group_by(&:row).map do |key, value|
    row_at(
      tickets: value.sort_by(&:number),
      number: key.to_i,
      tickets_to_buy_count: tickets_to_buy_count
    )
  end.sort_by(&:number)
end

def row_at(tickets:, number:, tickets_to_buy_count: 1)
  Row.new(
    tickets: tickets,
    number: number,
    tickets_to_buy_count: tickets_to_buy_count
  )
end

def tickets_in_row(row)
  tickets.select { |ticket| ticket.row == row }.sort_by(&:number)
end
```

This creates Row objects for each row of tickets in the concert, giving each row